



Table of Contents

Commands

Cypress commands don't do anything at the moment they are invoked, but rather enqueue themselves to be run later. Commands can be chained together because Cypress manages a Promise chain on your behalf, with each command yielding a 'subject' to the next command, until the chain ends or an error is encountered.

Learn more about how commands chain together in the [Introduction to Chains of Commands](#).

Cypress commands can be classified one of the following

- `query` - commands that read the state of your application
- `assertion` - command which assert on a given state
- `action` - interact with your application as a user would
- `other` - any other command that is useful for writing tests

The differences between queries and other commands are outline in the [Retry-ability Guide](#).

Queries

Queries are Cypress commands that read the state of your application. They return a subject for further commands to act or assert on, and retry as needed to make sure the DOM element or other data they yield is always up to date.

Command	Usage
<code>.as()</code>	Assign an alias for later use. Reference the alias later within a <code>cy.get()</code> query or <code>cy.wait()</code> command.



Command	Usage
<code>.children()</code>	Get the children of each element within a set of DOM elements.
<code>.closest()</code>	Get the first ancestral element that matches a selector.
<code>.contains()</code>	Select a DOM element by text content.
<code>.document()</code>	Get the <code>window.document</code> of the active page.
<code>.eq()</code>	Select a DOM element by index from a collection.
<code>.filter()</code>	Filter elements with a selector.
<code>.find()</code>	Find descendent elements with a selector.
<code>.first()</code>	Select the first item in a collection.
<code>.focused()</code>	Get the DOM element that is currently focused.
<code>.get()</code>	Find DOM elements by selector, or read an alias previously created with the <code>.as()</code> command.
<code>.hash()</code>	Get the URL hash of the active page.
<code>.invoke()</code>	Invoke a function on the previously yielded subject.
<code>.its()</code>	Get a property's value on the previously yielded subject.
<code>.last()</code>	Select the last item in a collection.
<code>.location()</code>	Get the <code>window.location</code> object of the active page.
<code>.next()</code>	Get the next sibling element.



Command	Usage
<code>.nextAll()</code>	Get all following sibling elements.
<code>.nextUntil()</code>	Get all following sibling elements until reaching a selector.
<code>.not()</code>	Filter selected elements with a selector.
<code>.parent()</code>	Get the parent of a DOM element.
<code>.parents()</code>	Get all parent elements of a DOM element.
<code>.parentsUntil()</code>	Get all parent elements up to a selector.
<code>.prev()</code>	Get the previous sibling element.
<code>.prevAll()</code>	Get all previous sibling elements.
<code>.prevUntil()</code>	Get all following previous sibling elements until reaching a selector.
<code>.root()</code>	Get the root DOM element.
<code>.shadow()</code>	Traverse into the shadow DOM of an element.
<code>.siblings()</code>	Get all sibling elements.
<code>.title()</code>	Get the <code>document.title</code> property of the active page.
<code>.url()</code>	Get the URL of the active page.
<code>.window()</code>	Get the <code>window</code> object of the active page.

Assertions



Assertions are Cypress commands that assert on the state of your application. They pause your test until the given condition is met, or until they time out. Learn more about Cypress assertions in the [Introduction to Assertions](#) and the [Assertions Reference](#).

Command	Usage
<code>.and()</code>	Alias for <code>.should()</code> . Assert on the state of your application.
<code>.should()</code>	Assert on the state of your application. Assertions are automatically retried until they pass or time out.

Actions

Actions are Cypress commands that interact with your application as a user would. They wait for elements to be actionable before interacting with the page.

Learn more about action commands in our [Actionability Guide](#).

Command	Usage
<code>.check()</code>	Check checkbox or radio elements.
<code>.clear()</code>	Clear the value of an input or textarea.
<code>.click()</code>	Click a DOM element.
<code>.dblclick()</code>	Double-click a DOM element.
<code>.rightclick()</code>	Right click a DOM element.
<code>.scrollIntoView()</code>	Scroll an element into view.
<code>.scrollTo()</code>	Scroll to a specific position.
<code>.select()</code>	Select an <code><option></code> within a <code><select></code> .



Command	Usage
<code>.selectFile()</code>	Selects a file in an HTML5 input element, or simulates dragging a file into the browser.
<code>.trigger()</code>	Trigger an event on a DOM element.
<code>.type()</code>	Type into a DOM element.
<code>.uncheck()</code>	Uncheck checkbox(es).

Other Commands

Cypress has a variety of additional commands to help write tests.

Command	Usage
<code>.blur()</code>	Blur a focused element. Blur is not an action command, because users cannot directly unfocus an element, only focus a new one, but <code>.blur()</code> can be used as a testing shortcut.
<code>.clearAllCookies()</code>	Clear all browser cookies.
<code>.clearAllLocalStorage()</code>	Clear <code>localStorage</code> data for all origins with which the test has interacted.
<code>.clearAllSessionStorage()</code>	Clear <code>sessionStorage</code> data for all origins with which the test has interacted.
<code>.clearCookie()</code>	Clear a specific browser cookie.
<code>.clearCookies()</code>	Clear browser cookies for a domain.
<code>.clearLocalStorage()</code>	Clear data in <code>localStorage</code> for current domain and subdomain.



Command	Usage
<code>.clock()</code>	Override the browser's built-in time objects (<code>Date</code> , <code>setTimeout</code> and similar), allowing them to be controlled via <code>cy.tick()</code> .
<code>.debug()</code>	Set a <code>debugger</code> statement and log what the previous command yields.
<code>.each()</code>	Invoke a callback on each item in an array.
<code>.end()</code> deprecated	Explicitly end a chain of Cypress commands.
<code>.env()</code>	Access secret environment variables in your tests.
<code>.exec()</code>	Execute a system command.
<code>.fixture()</code>	Load a fixed set of data from disk.
<code>.focus()</code>	Focus on a DOM element. Focus is not an action command, because users cannot directly focus an element, but <code>.focus()</code> can be used as a testing shortcut.
<code>.getAllCookies()</code>	Get all browser cookies.
<code>.getAllLocalStorage()</code>	Get <code>localStorage</code> data for all origins with which the test has interacted.
<code>.getAllSessionStorage()</code>	Get <code>sessionStorage</code> data for all origins with which the test has interacted.
<code>.getCookie()</code>	Get a browser cookie by its name.



Command	Usage
<code>.getCookies()</code>	Get browser cookies for the current domain or a specified domain.
<code>.go()</code>	Navigate back or forward using the browser's history.
<code>.hover()</code>	Cypress does not have a <code>cy.hover()</code> command. See Issue #10 .
<code>.intercept()</code>	Spy and stub network requests and responses.
<code>.log()</code>	Print a message to the Cypress Command Log.
<code>.mount()</code>	Mount a component for Cypress Component Testing.
<code>.origin()</code>	Visit multiple domains of different origin in a single test.
<code>.pause()</code>	Pause test execution, allowing interaction with the application under test before resuming.
<code>.press()</code>	Trigger native key events in your application to simulate real user keyboard interactions.
<code>.prompt()</code>	Use natural language to generate Cypress tests with AI.
<code>.readFile()</code>	Read a file from disk.
<code>.reload()</code>	Reload the page.
<code>.request()</code>	Make an HTTP request.



Command	Usage
<code>.screenshot()</code>	Take a screenshot of the application under test.
<code>.session()</code>	Cache and restore cookies, localStorage, and sessionStorage (i.e. session data) in order to recreate a consistent browser context between tests.
<code>.setCookie()</code>	Set a browser cookie.
<code>.spread()</code>	Invoke a callback function with multiple arguments. Similar to <code>.then()</code> .
<code>.spy()</code>	Wrap a method in a spy in order to record calls to the function.
<code>.stub()</code>	Replace a method, record its usage and control its behavior.
<code>.submit()</code>	Submit a form.
<code>.task()</code>	Execute code in Node via the <code>task</code> plugin event.
<code>.then()</code>	Invoke a callback function with the current subject, allowing you to execute code at specific points during a Cypress test.
<code>.tick()</code>	Move time after overriding a native time functions with <code>cy.clock()</code> .
<code>.viewport()</code>	Control the size and orientation of the screen for your application.
<code>.visit()</code>	Visit a remote URL. Many tests begin with this command.



Command	Usage
<code>.wait()</code>	Wait a number of milliseconds, or wait for an aliased resource to resolve.
<code>.within()</code>	Scopes all subsequent cy commands to within this element. Useful when working within a particular group of elements such as a <code><form></code> .
<code>.wrap()</code>	Yield the object passed into <code>.wrap()</code> . If the object is a promise, yield its resolved value.
<code>.writeFile()</code>	Write to a file with the specified contents.

Cypress API

Events

Cypress emits a series of events as it runs in your browser. These events are useful not only to control your application's behavior, but also for debugging purposes. Learn more about them in the [Catalog of Events](#).

Custom Commands and Queries

Cypress exposes interfaces to write [Custom Commands](#) and [Custom Queries](#). See those pages for more details.

The Cypress community has created a large number of [Command Plugins](#) which you can install or download.

APIs

Cypress exposes a number of properties and methods on the global `Cypress` object.



The *key* difference between Cypress APIs and Cypress commands is that Cypress APIs execute the moment they are invoked and are **not** enqueued to run at a later time.

Property	Usage
<code>Cypress.arch</code>	CPU architecture name of the underlying OS, as returned from Node's <code>os.arch()</code> .
<code>Cypress.browser</code>	Information about the current browser, such as browser family and version.
<code>Cypress.Commands</code>	Create new custom commands and extend or override existing ones.
<code>Cypress.config()</code>	Get and set Cypress configuration from inside your tests.
<code>Cypress.Cookies.debug()</code>	Generate console logs whenever a cookie is modified.
<code>Cypress.currentRetry</code>	A number representing the current test retry count.
<code>Cypress.currentTest</code>	An object with information about the currently executing test.
<code>Cypress.log</code>	This is the internal API for controlling what gets printed to the Command Log. Useful when writing your own custom commands.
<code>Cypress.dom</code>	A collection of DOM related helper methods.
<code>Cypress.ElementSelector</code>	Configure selector priority used by Cypress Studio and cy.prompt() .



Property	Usage
<code>Cypress.env</code> deprecated	Get environment variables from inside your tests.
<code>Cypress.expose</code>	Expose configuration values to the browser context.
<code>Cypress.isBrowser()</code>	Checks if the current browser matches the given name or filter.
<code>Cypress.isCy()</code>	checks if a variable is a valid instance of cy or a cy chainable.
<code>Cypress.Keyboard.defaults()</code>	Set default values for how the <code>.type()</code> command is executed.
<code>Cypress.platform</code>	The underlying OS name, as returned by Node's <code>os.platform()</code> .
<code>Cypress.require</code>	Enables utilizing dependencies within the <code>cy.origin()</code> callback function.
<code>Cypress.Screenshot.defaults()</code>	Set defaults for screenshots captured by the <code>.screenshot()</code> command and the automatic screenshots taken during test failures.
<code>Cypress.session</code>	A collection of helper methods related to the <code>.session()</code> command.
<code>Cypress.spec</code>	An object with information about the currently executing spec file.
<code>Cypress.testingType</code>	The current testing type, eg. <code>"e2e"</code> or <code>"component"</code> .



Property	Usage
<code>Cypress.version</code>	The current Cypress version.

Utilities

Cypress makes several common libraries available directly on the global `Cypress` object.

Utility	Usage
<code>Cypress.\$</code>	Cypress automatically includes jQuery and exposes it as <code>Cypress.\$</code> .
<code>Cypress.Blob</code>	Cypress automatically includes a Blob library and exposes it as <code>Cypress.Blob</code> .
<code>Cypress.Buffer</code>	Cypress automatically includes a Buffer polyfill for the browser and exposes it as <code>Cypress.Buffer</code> .
<code>Cypress._</code>	Cypress automatically includes lodash and exposes it as <code>Cypress._</code> .
<code>Cypress.minimatch</code>	Cypress automatically includes minimatch and exposes it as <code>Cypress.minimatch</code> .
<code>Cypress.Promise</code>	Cypress automatically includes Bluebird and exposes it as <code>Cypress.Promise</code> .
<code>Cypress.sinon</code>	Cypress automatically includes Sinon.JS and exposes it as <code>Cypress.sinon</code> .

Node Events

The Node Events allow you to hook into and extend Cypress behavior. For more details, see the [How to use a Plugin](#) and [Node Events Overview](#).



API	Usage
After Run Event	The <code>after:run</code> event fires after a run is finished. When running cypress via cypress open, the event will fire when closing a project.
After Screenshot Event	After a screenshot is taken, you can get details about it via the <code>after:screenshot</code> plugin event.
After Spec Event	The <code>after:spec</code> event fires after a spec file is run. When running cypress via cypress open, the event will fire when the browser closes.
Before Run Event	The <code>before:run</code> event fires before a run starts. When running cypress via cypress open, the event will fire when opening a project.
Before Spec Event	The <code>before:spec</code> event fires before a spec file is run. When running cypress via cypress open, the event will fire when the browser launches.
Browser Launch API	Before Cypress launches a browser, it gives you the opportunity to modify the browser preferences, install extensions, add and remove command-line arguments, and modify other options from the <code>setupNodeEvents</code> function.
Configuration API	Cypress enables you to dynamically modify configuration values and environment variables from your Cypress configuration.
Preprocessors API	A preprocessor is the plugin responsible for preparing a support file or a test file for the browser. A preprocessor also typically watches the source files for changes, processes them again, and then notifies Cypress to re-run the tests.



*Last updated on **May 12, 2026***
CONTENTS

